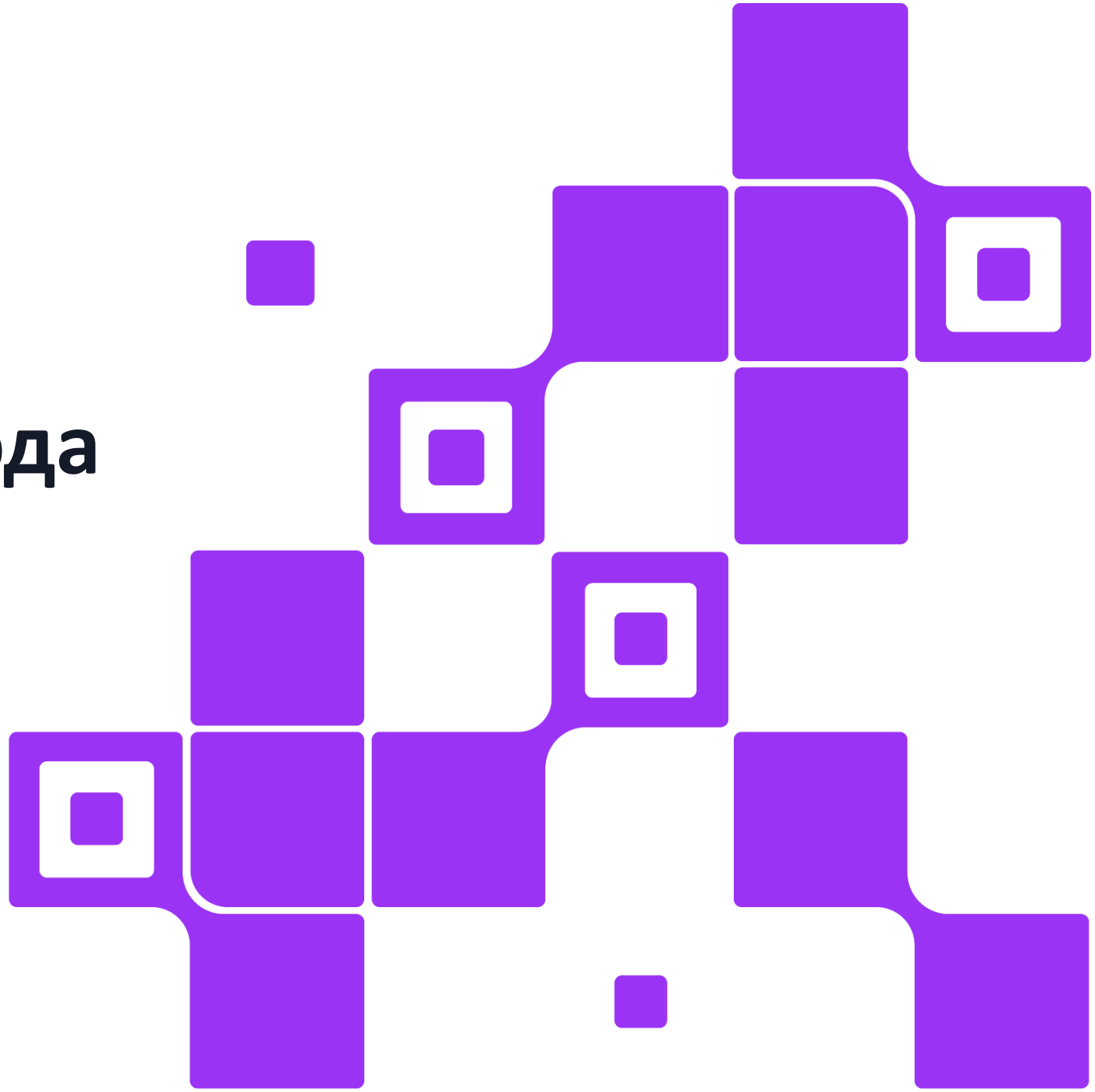
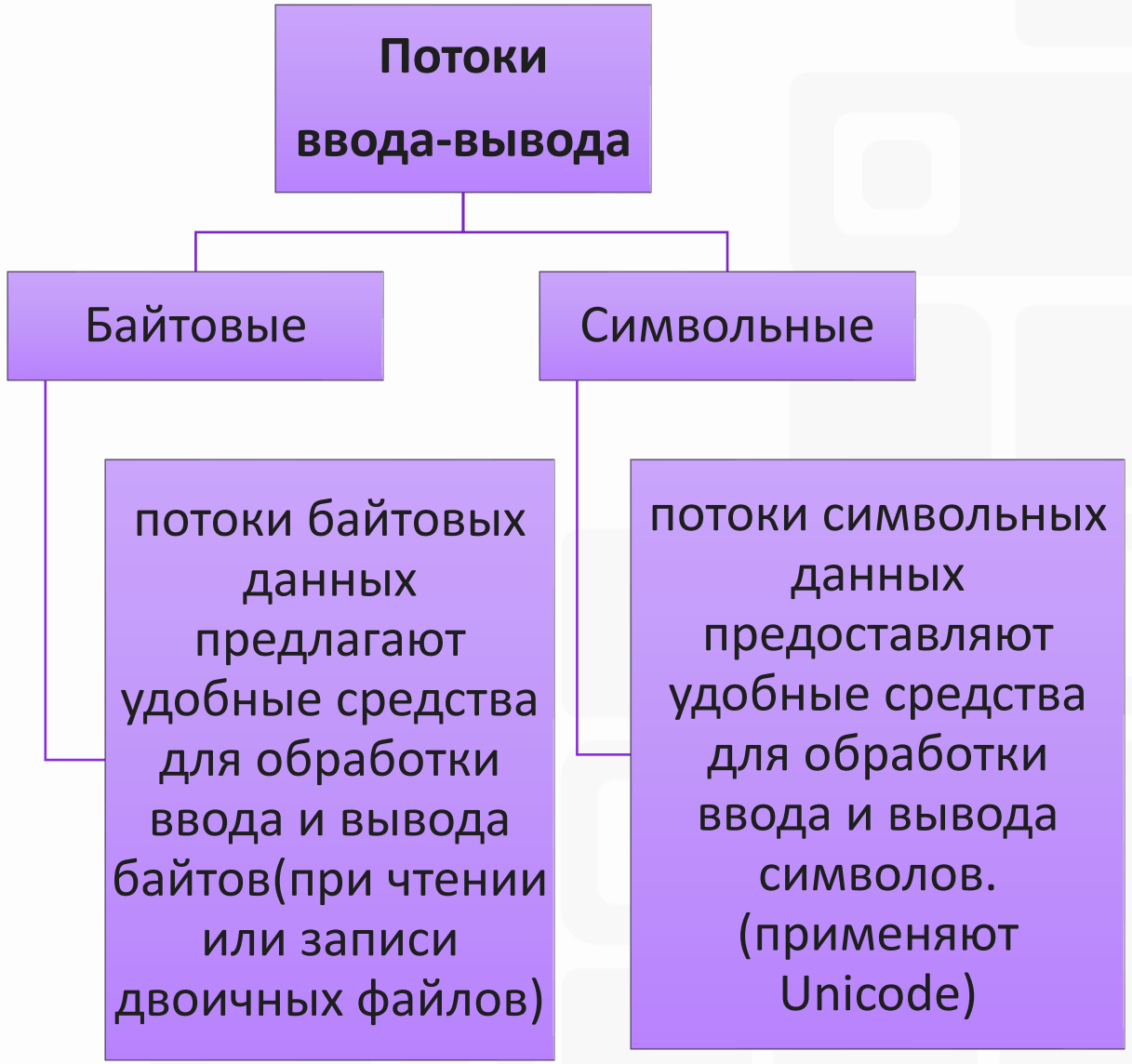
Потоки ввода/вывода



IThub

Потоки данных

- Ввод-вывод в программах на Java выполняется через потоки данных.
- **Поток данных** (stream) - это абстракция, которая либо производит, либо потребляет информацию. Поток связан с физическим устройством посредством системы ввода-вывода Java (из дискового файла, клавиатуры или сетевого сокета)

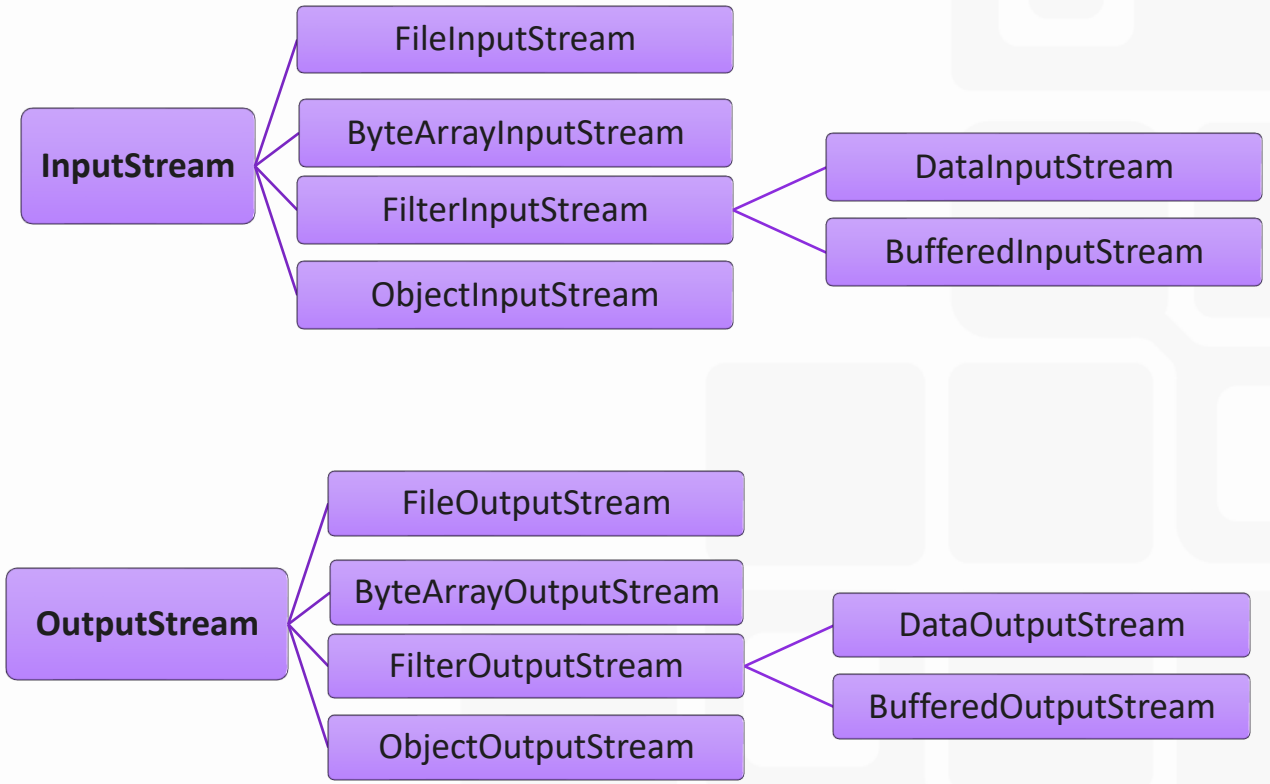


Классы потоков байтовых данных

IThub

В абстрактных классах **InputStream** и **OutputStream** определено несколько ключевых методов, реализуемых другими классами потоков.

Двумя наиболее важными из них являются **read()** и **write()**, которые выполняют, соответственно, чтение и запись байтов данных. У каждого имеется абстрактная форма, которая должна быть переопределена в производных классах потоков



InputStream

```
abstract class InputStream extends Object implements Closeable
```

- абстрактный класс, определяющий модель потокового байтового ввода в Java

```
class FileInputStream extends InputStream
```

- создает объект InputStream, который можно применять для чтения байтов из файла

```
class ByteArrayInputStream extends InputStream
```

- реализация потока ввода, использующая в качестве источника байтовый массив

```
class BufferedInputStream extends FilterInputStream
```

- позволяет помещать любой объект InputStream в оболочку буферизованного потока с целью повышения производительности (создает буферизированный поток с размером буфера)

OutputStream

```
abstract class OutputStream extends Object implements Closeable, Flushable
```

- абстрактный класс, определяющий потоковый вывод байтов

```
class FileOutputStream extends OutputStream
```

- создает объект OutputStream, который можно применять для записи байтов в файл

```
class ByteArrayOutputStream extends OutputStream
```

- реализация потока вывода, который использует байтовый массив в качестве места назначения

```
class BufferedOutputStream extends FilterOutputStream
```

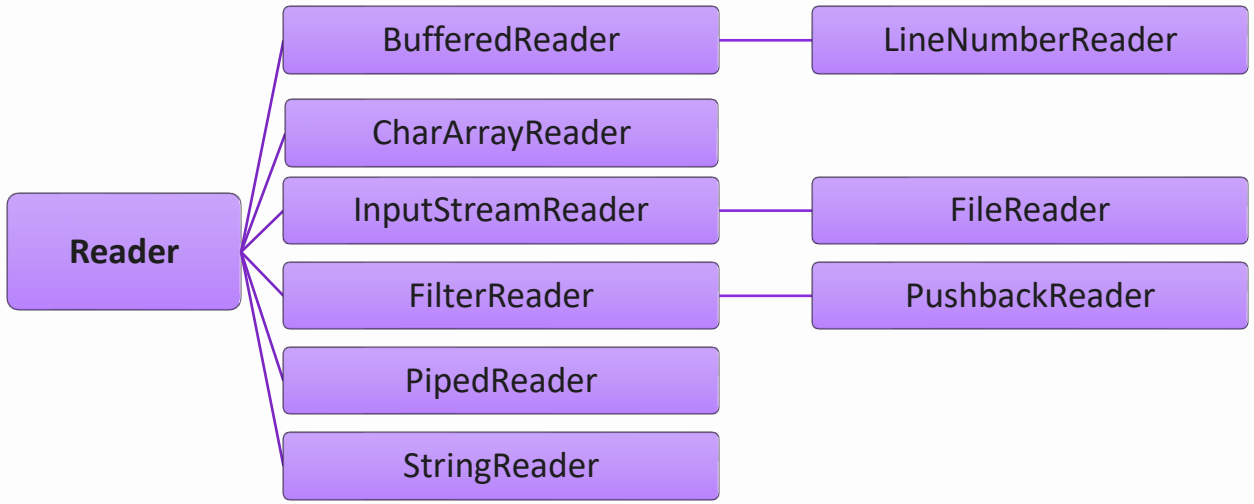
- похож на любой OutputStrearn за исключением того, что метод flush() применяется для обеспечения записи буферов данных в буферизуемый поток

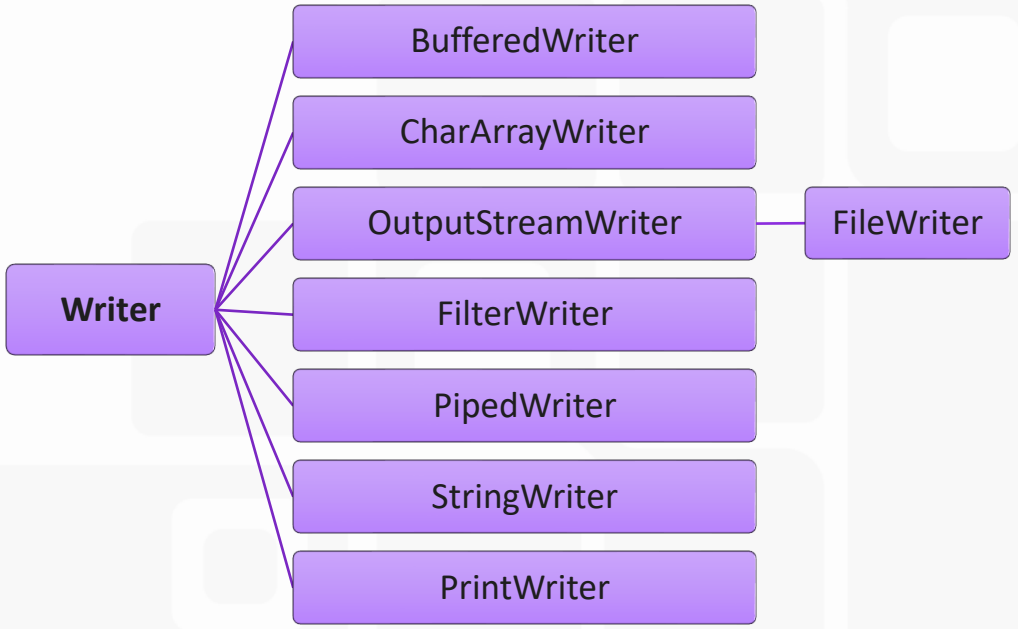
Классы DataOutputStream и DataInputStream позволяют записывать данные примитивных типов в поток или читать их из него. Они реализуют соответственно интерфейсы DataOutput и DataInput, которые определяют методы, преобразующие значения примитивных типов в последовательность байтов либо из нее. Эти потоки упрощают хранение двоичных данных, таких как целые числа или значения с плавающей точкой, в файле

Классы потоков символьных данных

Потоки символьных данных определяются с помощью двух иерархий классов. Вверху находятся два абстрактных класса: Reader и Writer. Они обрабатывают потоки символов Unicode.

Аналогично байтовым данным, двумя наиболее важными методами считаются **read()** и **write()**, которые выполняют, соответственно, чтение и запись символов данных





Reader

```
abstract class Reader extends Object implements Readable, Closeable
```

- абстрактный класс, определяющий модель потокового ввода символов в Java

```
class FileReader extends InputStreamReader
```

- создает средство чтения, которое можно применять для чтения содержимого файла

```
class CharArrayReader extends Reader
```

- реализация потока ввода, которая применяет в качестве источника символьный массив

```
class BufferedReader extends Reader
```

- повышает производительность за счет буферизации ввода

```
class PushbackReader extends FilterReader
```

- позволяет возвращать во входной поток один или несколько символов, что позволяет заглядывать вперед во входном потоке

Writer

```
abstract class Writer extends Object implements Appendable, Closeable, Flushable
```

- абстрактный класс, определяющий модель потокового вывода символов в Java

```
class FileWriter extends OutputStreamWriter
```

- создает объект Writer, который можно применять для записи в файл

```
class CharArrayWriter extends Writer
```

- реализация потока вывода, который использует массив в качестве места назначения

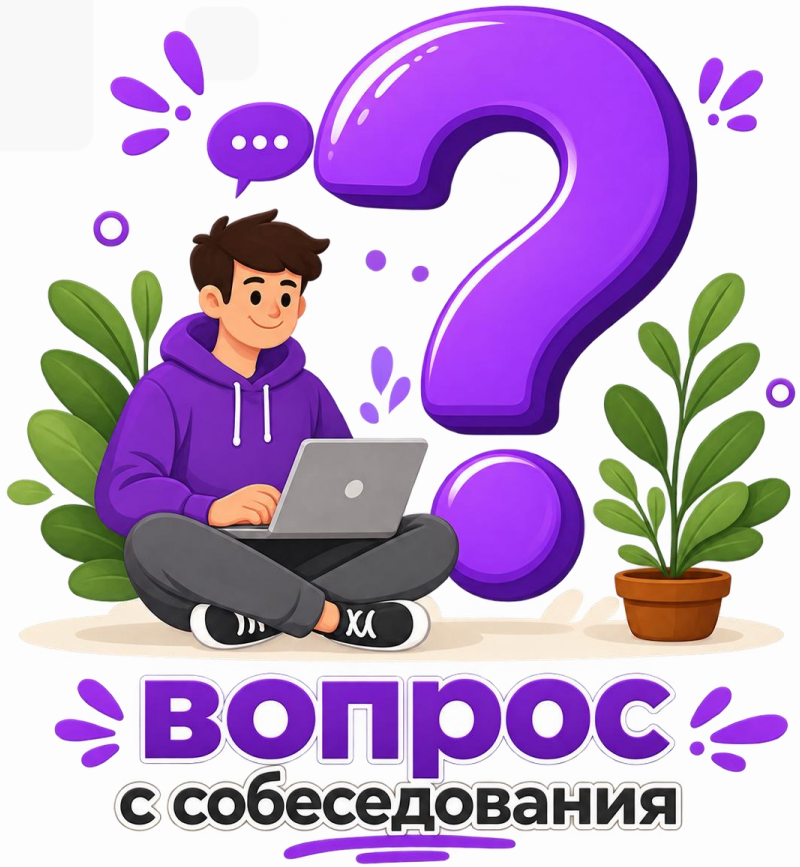
```
class BufferedWriter extends Writer
```

- представляет собой Writer с буферизацией вывода. За счет его использования можно повысить производительность, уменьшая фактическое количество операций физической записи данных на устройство вывода

```
class PrintWriter extends Writer
```

- является по существу версией PrintStream, ориентированной на символы

IThub



Чем отличаются и что общего у InputStream, OutputStream, Reader, Writer?

InputStream и его наследники - совокупность для получения байтовых данных из различных источников;

OutputStream и его наследники - набор классов, определяющих потоковый байтовый вывод;

Reader и его наследники определяют потоковый ввод символов Unicode;

Writer и его наследники определяют потоковый вывод символов Unicode.

Чтение консольного ввода

IThub

- консольный ввод в Java выполняется путем чтения из System.in
- один из способов получения символьного потока, присоединенного к консоли, предусматривает помещение System.in в оболочку BufferedReader

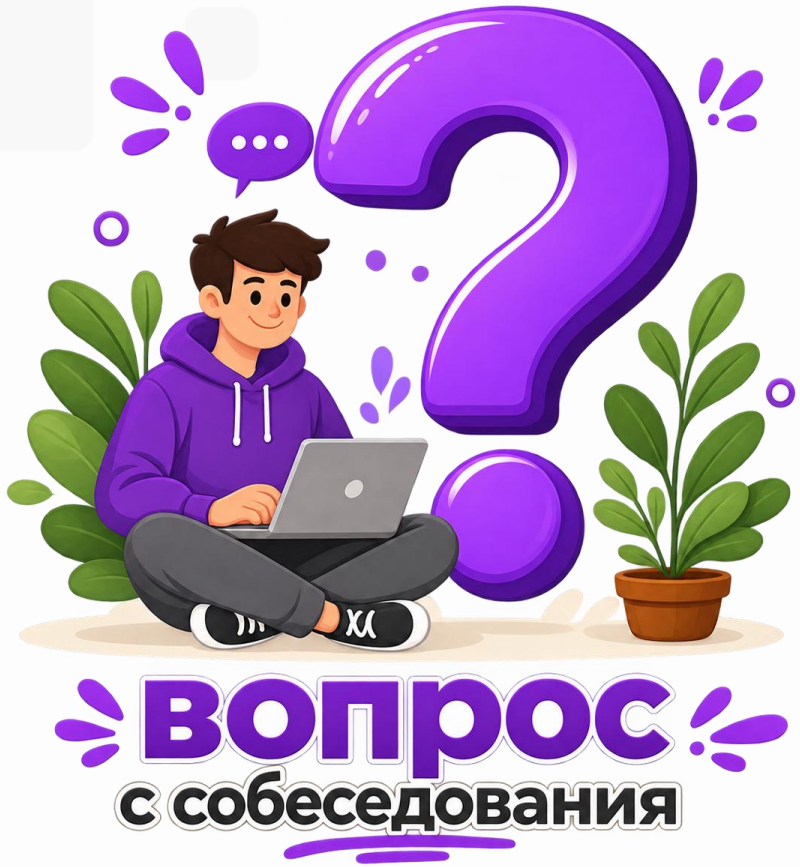
```
public void consoleReaderExample() throws IOException {
   BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
   StringBuilder str = new StringBuilder();

   System.out.println("Enter some symbols:");

   char c;
   // чтение символов пока не введено 'e'
   do {
      c = (char) br.read();
      str.append(c);
   } while (c != 'e');

   System.out.println("Symbols are: " + str);
}
```

IThub



Существует ли возможность перенаправить потоки стандартного ввода/вывода?

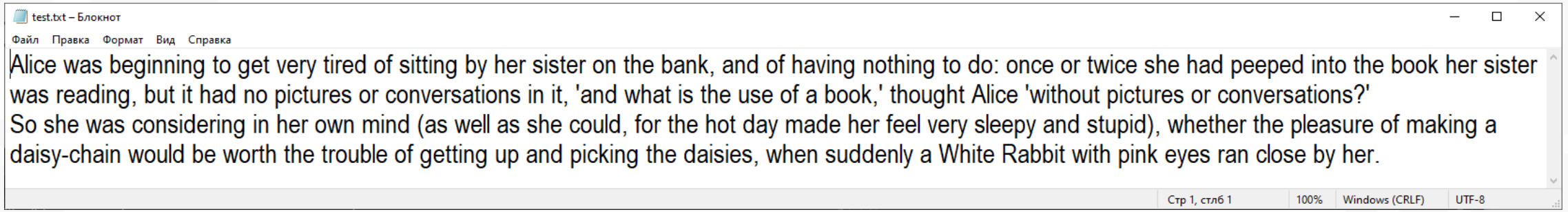
Класс System позволяет перенаправлять стандартный ввод, вывод и поток вывода ошибок, используя простой вызов статического метода:

- setIn(InputStream) - для ввода;
- setOut(PrintStream) - для вывода;
- setErr(PrintStream) - для вывода ошибок.

Чтение файлов и запись в файлы

```
FileInputStream(String fileName) throws FileNotFoundException
FileOutputStream(String fileName) throws FileNotFoundException
```

- создают потоки байтовых данных, связанные с файлами
- в аргументе fileName указывается имя файла, который необходимо открыть. Если при создании входного потока файл не существует, тогда генерируется исключение FileNotFoundException
- завершив работу с файлом, необходимо его закрыть, что делается вызовом метода close(). Также можно использовать try с ресурсами, который автоматически закрывает файл




```java
public void fileReaderExample() {
    System.out.println("Reading file: " + fileName + "\n");

    FileInputStream fileInputStream;
    int i;

    // попытка открыть файл
    try {
        fileInputStream = new FileInputStream(fileName);
    } catch (FileNotFoundException e) {
        System.out.println("File not found");
        return;
    }
```

```java
// читаем символы до тех пор, пока не встретится EOF
    try {
        do {
            i = fileInputStream.read();
            if (i != -1) {
                System.out.print((char) i);
            }
        } while (i != -1);
    } catch (IOException e) {
        System.out.println("Error reading file");
    }

    // закрываем файл
    try {
        fileInputStream.close();
    } catch (IOException e) {
        System.out.println("Error closing file");
    }
}
```

IThub

```java
public void fileReaderTryWithResourcesExample() {
    System.out.println("Reading file: " + fileName + "\n");
    StringBuilder text = new StringBuilder();

    try (InputStream fileInputStream = new FileInputStream(fileName)) {
        int data;
        do {
            data = fileInputStream.read();
            if (data != -1) {
                text.append((char) data);
            }
        } while (data != -1);
    } catch (IOException e) {
        System.out.println("Error reading file: " + e.getMessage());
    }

    System.out.println(text);
}
```

```
Reading file: src/main/java/org/example/io/test.txt

Alice was beginning to get very tired of sitting by her sister on the bank, and of having nothing to do:
So she was considering in her own mind (as well as she could, for the hot day made her feel very sleepy
```

КОПИРОВАНИЕ ФАЙЛА

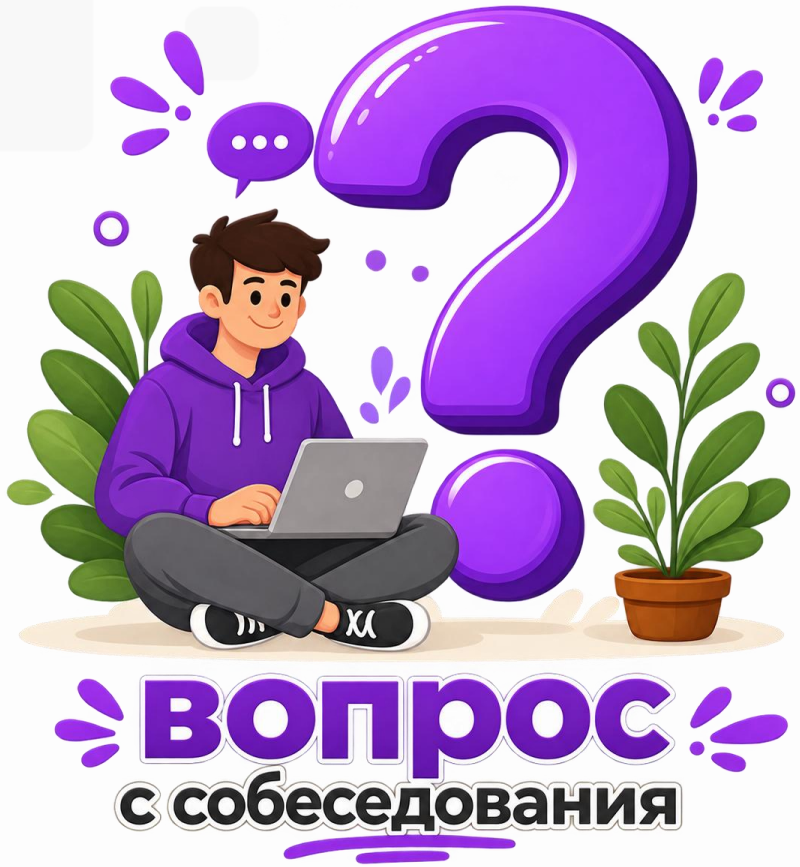
IThub

```
public void fileWriterExample() {
   System.out.println("Reading file: " + fileName + " and copy to: "  + copyFileName + "\n");

   int data;

   try (InputStream fileInputStream = new FileInputStream(fileName);
        OutputStream fileOutputStream = new FileOutputStream(copyFileName)) {
      do {
         data = fileInputStream.read();
         if (data != -1) {
            fileOutputStream.write(data);
         }
      } while (data != -1);
   } catch (IOException e) {
      System.out.println("Error copying file: " + e.getMessage());
   }
}
```

IThub



Какой класс предназначен для работы с элементами файловой системы?

File работает непосредственно с файлами и каталогами. Данный класс позволяет создавать новые элементы и получать информацию существующих: размер, права доступа, время и дату создания, путь к родительскому каталогу.

Класс File

IThub

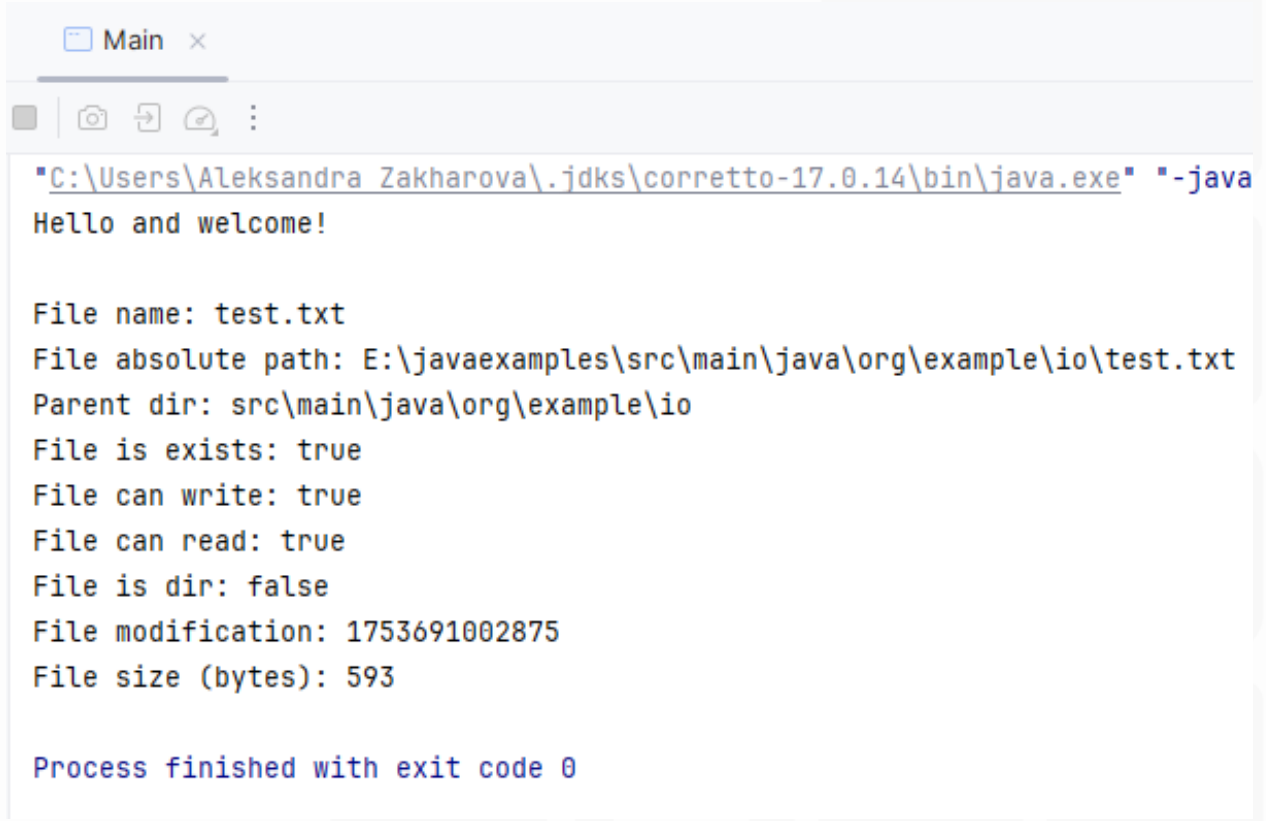
```
class File extends Object
```

- имеет дело напрямую с файлами и файловой системой
- применяется для получения или управления информацией, связанной с файлом на диске, такой как разрешения, время, дата и путь к каталогу, а также для навигации по иерархии подкаталогов

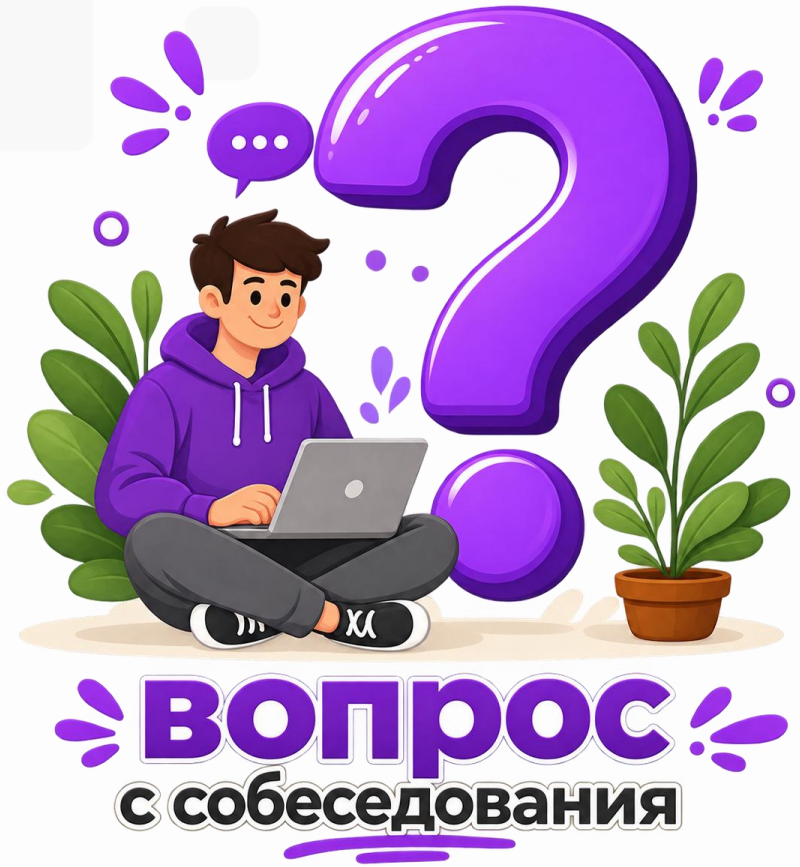
Модификатор и тип	Метод	Описание
boolean	delete()	Удаляет файл или каталог, обозначенный этим абстрактным путем
void	deleteOnExit()	Удаляет файл, ассоциированный с вызывающим объектом, когда виртуальная машина Java прекращает работу
boolean	exists()	Проверяет, существует ли файл или каталог
String	getAbsolutePath()	Возвращает строку абсолютного пути
long	getFreeSpace()	Возвращает количество свободных байтов в разделе, названном этим абстрактным путем
long	getTotalSpace()	Возвращает емкость раздела, ассоциированного с вызывающим объектом
boolean	isHidden()	Проверяет, является ли файл, названный этим абстрактным путем, скрытым файлом
long	lastModified()	Возвращает время последнего изменения файла, обозначенного этим абстрактным путем
boolean	setReadOnly()	Делает вызывающий файл доступным только для чтения

IThub

```java
public void fileClassDemo() {
    File file = new File(fileName);
    System.out.println("File name: " + file.getName());
    System.out.println("File absolute path: " + file.getAbsolutePath());
    System.out.println("Parent dir: " + file.getParent());
    System.out.println("File is exists: " + file.exists());
    System.out.println("File can write: " + file.canWrite());
    System.out.println("File can read: " + file.canRead());
    System.out.println("File is dir: " + file.isDirectory());
    System.out.println("File modification: " + file.lastModified());
    System.out.println("File size (bytes): " + file.length());
}
```



IThub



Что такое «абсолютный путь» и «относительный путь»?

Абсолютный (полный) **путь** — это путь, который указывает на одно и то же место в файловой системе, вне зависимости от текущей рабочей директории или других обстоятельств. Полный путь всегда начинается с корневого каталога.

Относительный путь представляет собой путь по отношению к текущему рабочему каталогу пользователя или активного приложения.

FileReader

```
import java.io.FileReader;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {

        try (FileReader reader = new FileReader("input.txt")) {

            int character;

            while ((character = reader.read()) != -1) {
                System.out.print((char) character);
            }

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Используется, чтобы читать текст из файла.

Метод read() возвращает

- код прочитанного символа
- -1, если файл закончился

FileWriter

FileWriter используется для записи текста в файл.

```
import java.io.FileWriter;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {

        try (FileWriter writer = new FileWriter("output.txt")) {

            writer.write("Hello Java!");

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

FileWriter **перезаписывает** файл!

Если файл output.txt уже существует по умолчанию его содержимое будет перезаписано.

Как добавить текст в конец?
Используем конструктор с true:

```
try (FileWriter writer = new FileWriter("output.txt", true)) {

    writer.write("\nNew line");

} catch (IOException e) {
    e.printStackTrace();
}
```

Теперь старое содержимое сохранится, а New line добавится в конец.

FileReader + FileWriter

IThub

Скопировать содержимое одного текстового файла в другой:

```
try (
    FileReader reader = new FileReader("input.txt");
    FileWriter writer = new FileWriter("output.txt")
) {

    int character;

    while ((character = reader.read()) != -1) {
        writer.write(character);
    }

} catch (IOException e) {
    e.printStackTrace();
}
```

Закрытие ресурса

Обратите внимание:

```
try (FileReader reader = new FileReader("input.txt")) {
    // работа с файлом
}
```

После завершения try Java автоматически вызовет метод **reader.close();**

То же самое для FileWriter.

Это важно, потому что **файл является ресурсом**, который нужно **закрывать** после использования. Можно было бы написать:

```
FileReader reader = new FileReader("input.txt");

try {
    // читаем
} finally {
    reader.close();
}
```

Задача (FileReader, FileWriter)

IThub

Анализатор заметок

У нас есть файл notes.txt:

```
Java — отличный язык программирования.
Сегодня изучаем FileReader и FileWriter.
Работа с файлами очень важна.
Java позволяет удобно работать с текстом.
```

Задание:

1. Используя FileReader, прочитайте содержимое notes.txt
    1. Посчитать количество символов while ch != -1
    2. Посчитать количество строк if (ch == '\n')
    3. Посчитать количество слов Character.isWhitespace(ch)

2. Используя FileWriter создайте файл report.txt

```
ОТЧЁТ
Количество строк: 4
Количество слов: 21
Количество символов: 156
```

3. Поиск слова

Пользователь вводит слово: Java

Программа должна определить, сколько раз оно встречается в файле и добавить эту информацию в report.txt